

Routing Strategies for Ring and General Topologies in OMNeT++

Gerbaudo, Nicolás Ignacio; Jorge González, Nicolás; Ontivero, Nahuel Mauricio; Vides, Alejo Miguel
Grupo 25, Facultad de Matemática, Astronomía, Física y Computación (FaMAF)
Universidad Nacional de Córdoba, Argentina

Abstract

This paper analyzes and compares two routing strategies for a simulated ring network of 8 nodes implemented in OMNeT++. The baseline algorithm, provided as a kickstarter, always forwards packets through `lnk[0]`, concentrating all traffic in one direction of the ring and leaving half the network capacity unused. We instrumented the model with five metrics: end-to-end delay, hop count, buffer occupancy, link utilization, and forwarded packets per node. Two traffic scenarios were evaluated: Scenario 1, where only two nodes transmit to a single destination, and Scenario 2, where seven nodes simultaneously transmit to the same destination with varying inter-arrival times. Results show that the baseline saturates quickly, reaching average delays of over 50 seconds in Scenario 1 and requiring an inter-arrival time of at least 7 seconds to achieve stability in Scenario 2. The proposed algorithm selects the shortest path in both ring directions and uses buffer occupancy as a tiebreaker, reducing average delay by 86.5% in Scenario 1 and significantly improving performance in Scenario 2 under moderate and low load conditions. No routing loops were observed, as the algorithm always advances packets toward the destination. Both algorithms remain vulnerable to node failures due to the absence of dynamic topology discovery mechanisms.

Index Terms— Network simulation, OMNeT++, routing, ring topology, performance evaluation, queueing, stability.

1 Introducción

Este artículo aborda el problema del enrutamiento en una red en anillo de 8 nodos simulada en OMNeT++. Cada nodo cuenta con una capa de aplicación (`app`), una capa de red (`net`) y dos capas de enlace (`lnk[0]` y `lnk[1]`), que conectan al nodo con sus dos vecinos. El objetivo es analizar el comportamiento del algoritmo de enrutamiento provisto por la cátedra, identificar sus limitaciones y evaluar el rendimiento del sistema bajo distintos escenarios de tráfico. El resto del artículo se organiza de la siguiente manera: la Sección 2 describe el escenario y las métricas, la Sección 3 analiza el baseline, la Sección 4 presenta el algoritmo propuesto, la Sección 5 compara ambos enfoques y la Sección 6 resume las conclusiones.

En el algoritmo del kickstarter, cada nodo evalúa si es el destino final del paquete recibido: de serlo, lo entrega a su capa de aplicación; de lo contrario, lo reenvía siempre por la interfaz `toLnk[0]`, sin considerar la distancia ni el estado de la red, lo que implica que el tráfico siempre se envía en sentido horario a lo largo del anillo hasta llegar al destino. En consecuencia, la mitad de la capacidad de la red queda sin utilizar y ciertos enlaces concentran todo el tráfico, mientras otros permanecen inactivos; así se genera saturación en los buffers y demoras crecientes. Consideramos que una estrategia es mejor cuando reduce la demora media, disminuye la ocupación de buffers y aprovecha ambos sentidos del

anillo sin generar bucles de enrutamiento. En este trabajo evaluamos el rendimiento del sistema mediante métricas de demora extremo a extremo, ocupación de buffers, utilización de enlaces, cantidad de saltos por paquete y paquetes en tránsito por nodo.

El algoritmo propuesto elige la ruta más corta en ambas direcciones del anillo y, en caso de empate, desempata por congestión consultando la ocupación de los buffers locales. Los resultados muestran mejoras significativas en demora y cantidad de saltos en la mayoría de los escenarios evaluados.

2 Escenario y modelo de simulación

2.1 Topología y configuración de tráfico

La red simulada consiste en un anillo de 8 nodos (`node[0]` a `node[7]`), donde cada nodo se conecta con sus dos vecinos mediante enlaces *full-duplex* de 1 Mbps y 100 μ s de retardo. Cada nodo cuenta con tres capas:

- una capa de aplicación (`app`), que genera y recibe paquetes;
- una capa de red (`net`), que decide por qué interfaz reenviar cada paquete; y
- dos capas de enlace (`lnk[0]` y `lnk[1]`), cada una conectada a uno de los vecinos.

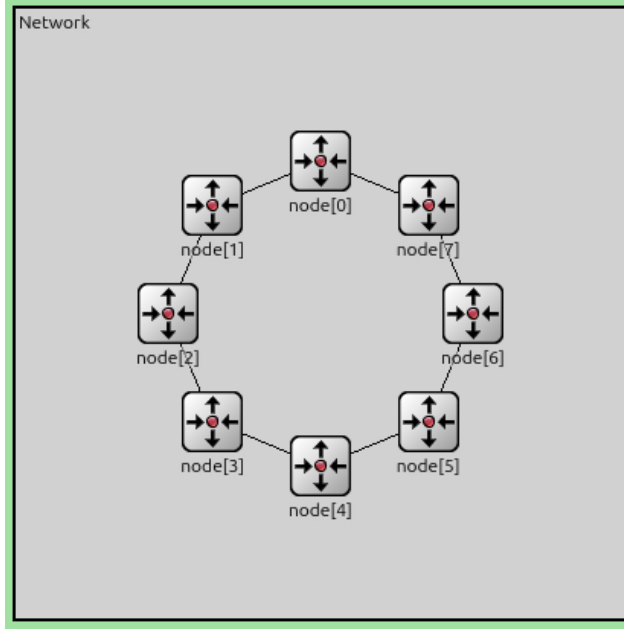


Figure 1: Topología en anillo de 8 nodos.

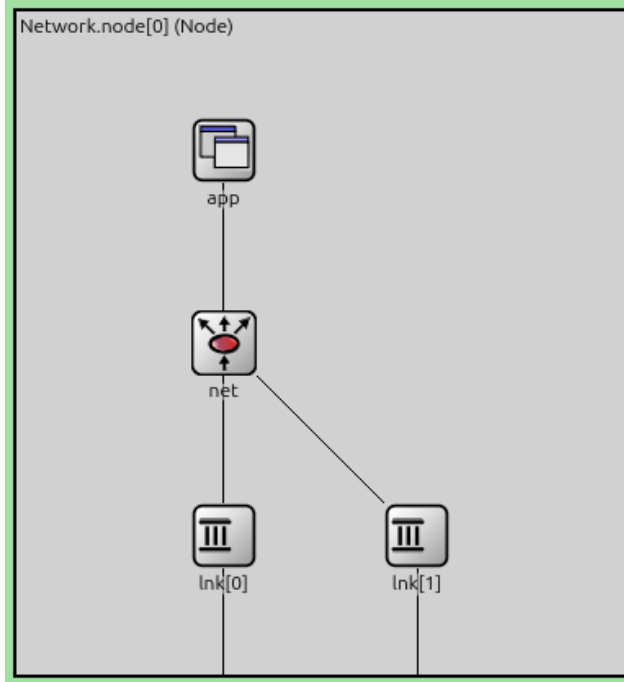


Figure 2: Estructura interna de `node[0]`: `lnk[0]` conecta con `node[7]` y `lnk[1]` conecta con `node[1]`.

La topología de la red y la estructura interna de un nodo se muestran en las Figs. 1 y 2, respectivamente.

Table 1: Configuración experimental por escenario.

Parámetro	Caso 1	Caso 2
Fuentes activas	n[0], n[2]	n[0-4], n[6-7]
Destino	n[5]	n[5]
packetByteSize	125000	125000
interArrivalTime	exp(1)	exp(0.5-10)
T. simulación (s)	200	200
Semilla	0 (default)	0 (default)

2.2 Métricas registradas

Las métricas se instrumentaron directamente en los módulos del modelo:

- **Demora extremo a extremo:** tiempo transcurrido desde que un paquete se crea en la `app` origen hasta que se recibe en la `app` destino. Se calcula como la diferencia entre el tiempo actual y el tiempo de creación del paquete, y se registra en `app` como vector (una muestra por paquete¹) y como escalar (promedio final).
- **Salto por paquete:** número de saltos que realiza un paquete desde su origen hasta su destino. El campo `hopCount` se incrementa en `net` en cada reenvío y se lee en `app` al recibir el paquete. Se registra como vector y como escalar (promedio final).
- **Ocupación de buffers:** cantidad de paquetes en espera en la cola de cada `lnk`. Se registra como vector cada vez que el tamaño de la cola cambia.
- **Utilización de enlaces:** fracción del tiempo de simulación durante la cual cada `lnk` estuvo transmitiendo activamente. Se acumula en `lnk` sumando el `serviceTime` de cada paquete transmitido y se reporta como escalar al final dividiendo por el tiempo total de simulación.
- **Paquetes en tránsito por nodo:** cantidad total de paquetes ajenos que cada `net` reenvió durante la simulación. Se cuenta en `net` como escalar.

¹Cada vez que llega un paquete se registra un punto en el vector, es decir, el vector tiene tantos puntos como paquetes recibidos.

Table 2: Métricas instrumentadas en el simulador.

Métrica	Módulo	Tipo	Unid.
Demora E2E	app	vector + escalar	s
Salto / paquete	app	vector + escalar	adim.
Ocupación buffer	lnk	vector	pkts
Utilización enlace	lnk	escalar	%
Paquetes en tránsito	net	escalar	adim.

3 Tarea de análisis: evaluación del kickstarter

3.1 Caso 1: tráfico configurado (node[0] y node[2] → node[5])

Configuración experimental. Sólo node[0] y node[2] transmiten hacia node[5] con `interArrivalTime` distribuido exponencialmente con media de 1 segundo y paquetes de 125000 bytes. El resto de los nodos no genera tráfico. La simulación corre durante 200 segundos.

Métricas obtenidas. Las métricas registradas durante la simulación se describen en la Sección 2.2.

Uso de recursos. Los recursos de la red se utilizan de forma muy desigual. Dado que el algoritmo del kickstarter reenvía todos los paquetes por `lnk[0]` (sentido horario), sólo 5 de las 8 interfaces `lnk[0]` transportan paquetes, mientras que todas las interfaces `lnk[1]` permanecen inactivas.

El enlace más cargado es `node[0].lnk[0]`, con una utilización del 99.5% y una ocupación media de buffer de 92.92 paquetes (Tabla 3), lo que indica saturación sostenida. Esto se debe a que por este enlace pasan los paquetes generados por `node[0]` y `node[2]`. La Fig. 3 confirma que el buffer de `node[0].lnk[0]` crece de forma sostenida a lo largo de la simulación, mientras que el de `node[2].lnk[0]` se mantiene relativamente acotado.

Los nodos intermediarios `node[1]`, `node[6]` y `node[7]` muestran una ocupación de buffer máxima de 1 paquete (Fig. 4), a pesar de utilizar sus enlaces entre el 93.5% y el 99%. Esto refleja que los paquetes fluyen de forma casi continua pero sin acumulación significativa en esos nodos.

En cuanto a los paquetes reenviados (Fig. 6), `node[6]` y `node[7]` reenvían aproximadamente 197 y 198 paquetes respectivamente. Dado que ambas fuentes generan aproximadamente 200 paquetes cada una y ambos nodos están en la ruta de las dos fuentes, se esperaría que reenviaran unos 400 paquetes. El hecho de que sólo hayan reenviado unos 200 es evidencia adicional de saturación: gran parte de los paquetes quedaron atrapados en los buffers al finalizar la simulación.

Table 3: Resultados del Caso 1 (baseline kickstarter).

Métrica	Prom.	Desv.
Demora extremo a extremo (s)	51.16	—
Salto por paquete	3.92	—
<i>Utilización de enlace (%)</i>		
node[0].lnk[0]	99.5	—
node[1].lnk[0]	93.5	—
node[2].lnk[0]	94.0	—
node[6].lnk[0]	98.5	—
node[7].lnk[0]	99.0	—
<i>Ocupación de buffer (paquetes)</i>		
node[0].lnk[0]	92.92	54.83
node[1].lnk[0]	0.50	0.50
node[2].lnk[0]	3.29	2.51
node[6].lnk[0]	0.50	0.50
node[7].lnk[0]	0.50	0.50
<i>Paquetes reenviados por nodo</i>		
node[0]	186	—
node[1]	187	—
node[6]	197	—
node[7]	198	—

El resto de los enlaces tiene utilización 0%.

Los `node[3].lnk[0]`, `node[4].lnk[0]` y `node[5].lnk[0]` no registraron ocupación de buffer por no haber tenido tráfico en sus colas durante la simulación.

El resto de los nodos reenvió 0 paquetes.

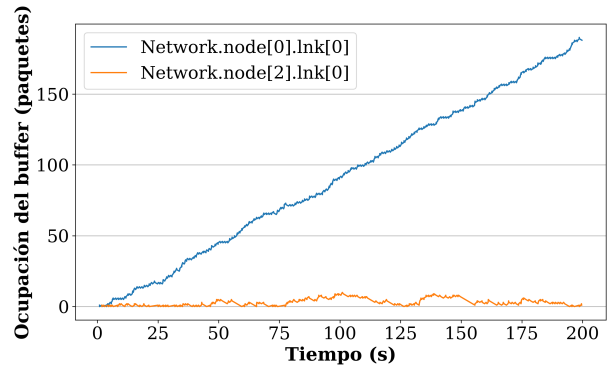


Figure 3: Ocupación de buffers a lo largo del tiempo — nodos fuente (node[0] y node[2]).

Posibilidades de mejora. El rendimiento puede mejorarse. El algoritmo actual ignora que cada nodo dispone de dos interfaces y que existen dos caminos posibles hacia cualquier destino. Al forzar siempre el uso de `lnk[0]`, se concentra todo el tráfico en un subconjunto de enlaces, dejando la mitad de la capacidad de la red completamente sin utilizar. Esto genera sat-

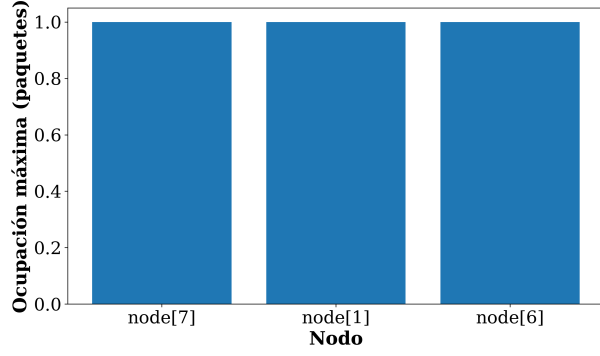


Figure 4: Ocupación máxima de buffers — nodos intermedarios (node[1], node[6] y node[7]).

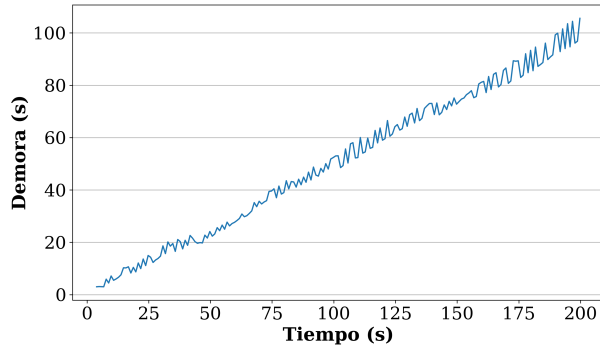


Figure 5: Demora extremo a extremo a lo largo del tiempo — node[5].

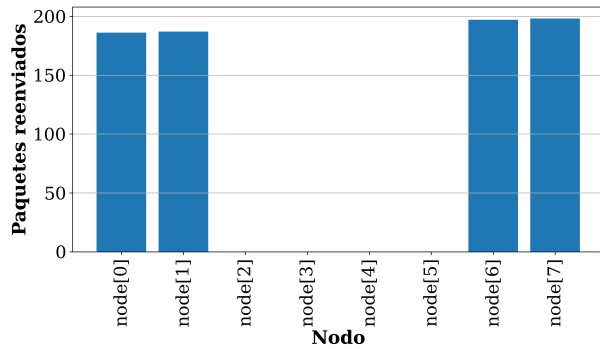


Figure 6: Paquetes reenviados por nodo durante la simulación.

uración en los enlaces más transitados y demoras crecientes, como se observa en la Fig. 5.

3.2 Caso 2: tráfico sostenido hacia node[5]

A partir de $\text{interArrivalTime} = 7\text{s}$ la red permanece estable porque los buffers de todos los nodos se mantienen acotados a lo largo de la simulación (Figs. 7–11 y Tabla 4). Para valores menores de interArrivalTime , al menos un buffer crece indefinidamente, lo que evidencia saturación.

La estabilización de la red a partir de $\text{iat}=7$ también se refleja en la demora media (Fig. 12), que cae drásticamente de 34s a 7s entre $\text{iat}=5$ y $\text{iat}=7$, y se mantiene en valores bajos para valores mayores de iat .

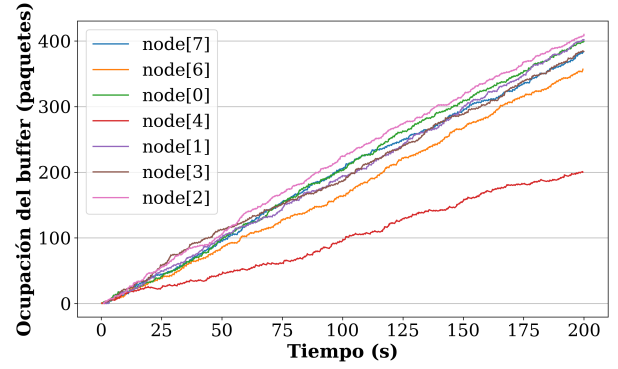


Figure 7: Ocupación de buffers a lo largo del tiempo — $\text{iat}=0.5\text{s}$.

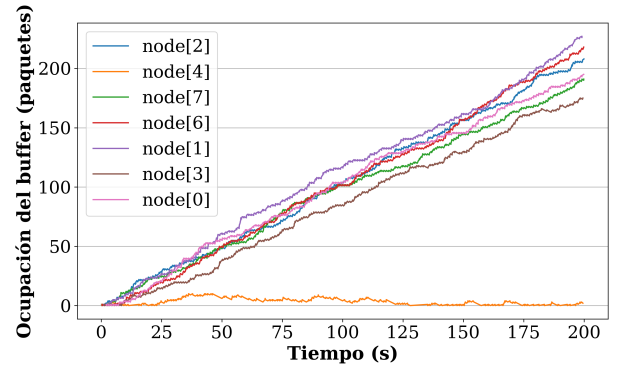


Figure 8: Ocupación de buffers a lo largo del tiempo — $\text{iat}=1\text{s}$.

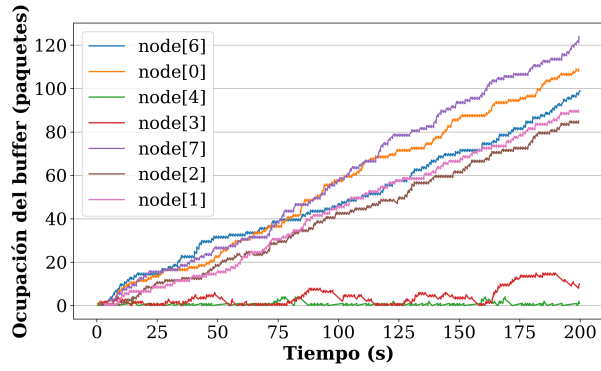


Figure 9: Ocupación de buffers a lo largo del tiempo — $iat=2s$.

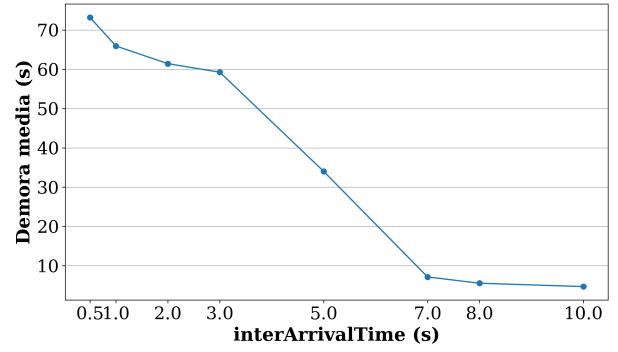


Figure 12: Demora media vs. `interArrivalTime` — Caso 2.

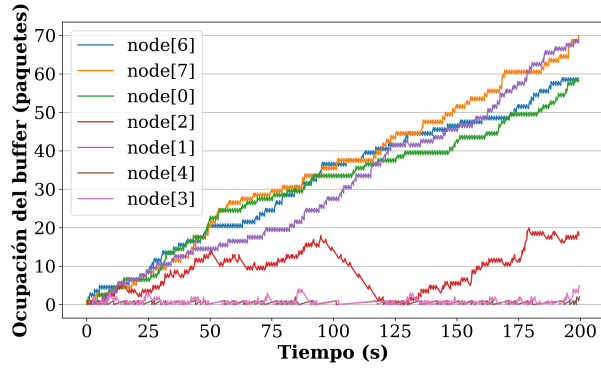


Figure 10: Ocupación de buffers a lo largo del tiempo — $iat=3s$.

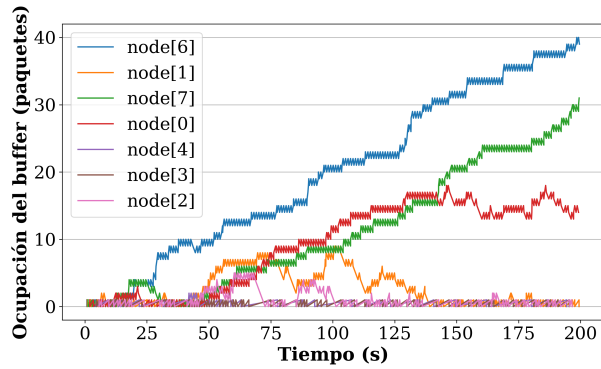


Figure 11: Ocupación de buffers a lo largo del tiempo — $iat=5s$.

Table 4: Exploración del Caso 2: métricas vs. `interArrivalTime`.

<code>interArriv.</code>	Demora (s)	Ocup. buffer máx.	Estab.
0.5	73.19	410	No
1.0	65.92	227	No
2.0	61.43	124	No
3.0	59.25	70	No
5.0	34.01	40	No
7.0	7.16	4	Sí
8.0	5.56	4	Sí
10.0	4.71	2	Sí

4 Diseño del algoritmo de enrutamiento propuesto

El algoritmo propuesto implementa un **enrutamiento consciente de la congestión** basado en el cálculo de distancias en ambas direcciones del anillo. A diferencia del baseline, que siempre elige `lnk[0]`, nuestro enfoque evalúa dinámicamente la mejor ruta considerando tanto la topología como el estado de los buffers.

4.1 Información disponible y estructuras de datos

Cada nodo dispone de la siguiente información **sin requerir paquetes de control adicionales**:

- **Topología local:** conoce su propio índice (`myIndex`) y el número total de nodos (`numNodes`). Esta información se obtiene durante la inicialización consultando el módulo padre.
- **Información del paquete:** el destino final (`pkt->getDestination()`) viaja en cada paquete desde su origen. No es necesario enviar mensajes hello/echo.
- **Estado local de buffers:** cada nodo accede a la longitud del buffer de sus dos interfaces (`lnk[0]` y `lnk[1]`) mediante el parámetro local `queueLength` del módulo `Lnk`.

Estas estructuras permiten que cada nodo tome decisiones de rutas locales sin requerir intercambio de información de control, lo que reduce la sobrecarga de la red.

4.2 Reglas de decisión

Descripción general. Al recibir un paquete, el nodo ejecuta el siguiente algoritmo:

1. Si el paquete está destinado al nodo actual, lo entrega a la capa de aplicación.
2. Si no es el destino, calcula la distancia más corta hacia el destino en ambas direcciones del anillo y elige la interfaz del camino más corto.
3. Si ambas distancias son iguales (empate), desempata usando el estado de congestión: elige la interfaz cuyo buffer tenga menor ocupación.
4. Reenvía el paquete por la interfaz seleccionada e incrementa el contador de saltos.

4.3 Comparación conceptual con el baseline

Table 5: Comparación conceptual: baseline vs. propuesto.

Aspecto	Baseline (Kickstarter)	Propuesto
Decisión de ruta	Siempre <code>lnk[0]</code>	Ambas direcciones
Métrica	Ninguna	Distancia + Congestión
Salto esperados	Hasta 7	Hasta 4 (promedio ≤ 4)
Equilibrio de carga	No	Sí
Demora típica (Caso 1)	≥ 50 s	Ver Sección 5
Utilización de enlaces	50% máximo	Hasta 100% distribuido

5 Evaluación comparativa

5.1 Metodología de comparación

Ambos algoritmos se ejecutaron con la misma configuración de red, parámetros y tiempos de simulación:

- **Topología:** anillo de 8 nodos, enlaces de 1 Mbps, 100 μ s de retardo.
- **Caso 1:** `node[0]` y `node[2]` generan tráfico hacia `node[5]` con `interArrivalTime` = `exponential(1)`, 125000 bytes/paquete, 200 s de simulación, semilla 0.
- **Caso 2:** `nodes [0–4]` y `[6–7]` generan tráfico hacia `node[5]`, barrido de `interArrivalTime` = {0.5, 1, 2, 3, 5, 7, 8, 10} segundos, 125000 bytes/paquete, 200 s cada simulación, semilla 0.
- **Métricas:** demora extremo a extremo (vector), saltos por paquete (vector), utilización de buffers.
- **Condiciones del propuesto:** requiere topología conocida (`numNodes`) en inicialización. No se usaron iteradores de `cGate` ni `cTopology` para descubrimiento dinámico. La información de destino viaja en cada paquete (sin hello/echo).

5.2 Resultados cuantitativos

La Fig. 13 resume la mejora más visible del algoritmo propuesto: en la mayor parte de los escenarios reduce de forma marcada la demora media, con una ganancia especialmente grande cuando la carga ofrecida permite aprovechar ambos sentidos del anillo.

5.3 Discusión de resultados

¿Por qué mejoran las métricas?

El propuesto elige el camino óptimo en ambas direcciones, además el desempate por congestión distribuye tráfico entre ambas direcciones. En enlaces con igual distancia se elige el buffer menos lleno. Es decir tenemos menos saltos, menos demora y al distribuir la carga, menos espera de buffers.

La Fig. 14 muestra la ocupación de buffers de los nodos fuente bajo el algoritmo propuesto. A diferencia

Table 6: Comparación baseline vs. algoritmo propuesto.

Escenario	Kickstarter (Baseline)		Propuesto		Mejora	
	Demora (s)	Salto	Demora (s)	Salto	Dem. (%)	Salto (%)
Caso 1 (node[0], node[2] → node[5])	51.16	3.92	6.90	3.00	−86.5%	−23.5%
Caso 2 — iat = 0.5 s	73.19	1.55	74.06	1.50	+1.2%	−3.2%
Caso 2 — iat = 1 s	65.92	1.96	64.16	1.81	−2.7%	−7.7%
Caso 2 — iat = 2 s	61.43	2.78	43.22	2.15	−29.7%	−22.7%
Caso 2 — iat = 3 s	59.25	3.35	20.14	2.26	−66.0%	−32.5%
Caso 2 — iat = 5 s	34.01	3.88	4.83	2.30	−85.8%	−40.7%
Caso 2 — iat = 7 s	7.16	3.86	2.78	2.34	−61.2%	−39.4%
Caso 2 — iat = 8 s	5.56	4.08	2.70	2.33	−51.4%	−42.9%
Caso 2 — iat = 10 s	4.71	4.00	2.58	2.35	−45.2%	−41.3%

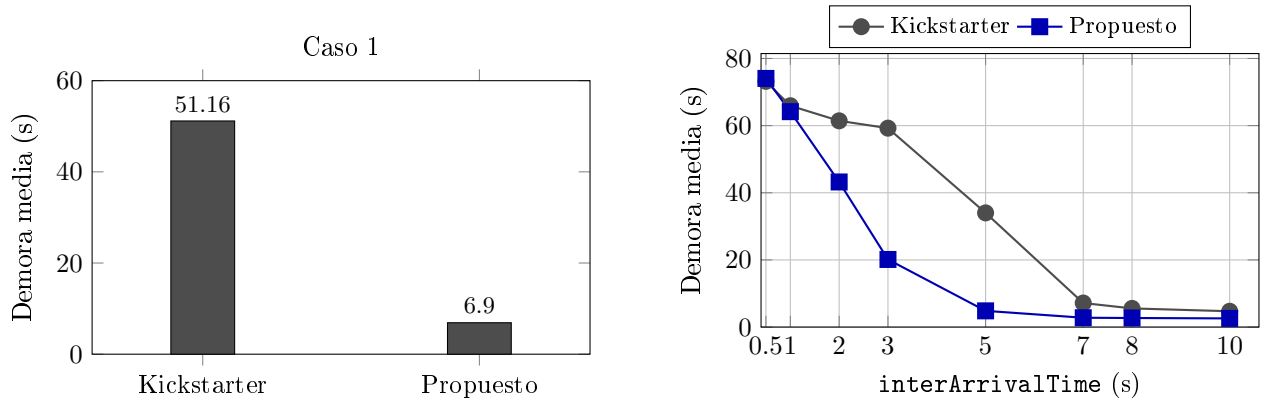


Figure 13: Comparación de la demora media entre el kickstarter y el algoritmo propuesto. A la izquierda se muestra el Caso 1 y a la derecha el barrido del Caso 2.

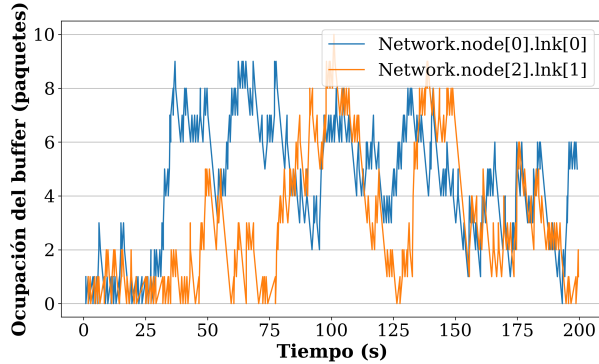


Figure 14: Ocupación de buffers a lo largo del tiempo — nodos fuente, algoritmo propuesto (Caso 1).

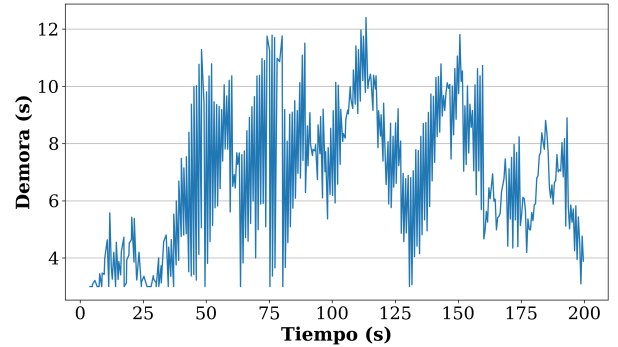


Figure 15: Demora extremo a extremo a lo largo del tiempo — algoritmo propuesto (Caso 1).

del baseline, donde el buffer de `node[0].lnk[0]` crecía de forma sostenida hasta superar los 190 paquetes, aquí ambos buffers permanecen acotados durante toda la simulación, con un máximo de 10 paquetes. Esto refleja que el uso de ambas direcciones del anillo distribuye

la carga y evita la saturación. La Fig. 15 confirma este comportamiento: la demora extremo a extremo se mantiene estable y acotada a lo largo de toda la simulación, en contraste con el crecimiento sostenido observado en el baseline.

¿Hubo bucles de enrutamiento?

Por la topología de anillo, al solo tener dos direcciones posibles y siempre elegir la más corta nunca nos vamos a encontrar con un bucle. En el peor caso, vamos a recorrer $n/2$ nodos para llegar al destino.

¿Qué pasaría con muchos más nodos? ¿Cómo escalaría respecto al baseline?

El propuesto escala mejor que el baseline porque: Utiliza ambas direcciones (capacidad de red $\times 2$ efectiva) y el calcular las distancias es $O(1)$. Por lo tanto, a medida que el número de nodos crece, la ventaja del propuesto se amplía, reduciendo la demora y evitando saturación en enlaces críticos, mientras que el baseline se vuelve cada vez más ineficiente al concentrar todo el tráfico en un solo sentido del anillo.

¿Qué sucedería si se cae un nodo? ¿Son tolerantes a fallos los algoritmos?

El algoritmo propuesto calcula rutas basándose en los índices, y en el caso de que un nodo falle no lo tendríamos en cuenta por lo que ambos son igualmente vulnerables a fallas de nodos críticos.

6 Conclusiones

El kickstarter concentra el tráfico en una sola dirección del anillo y, por eso, satura buffers y aumenta rápidamente la demora cuando la carga crece. En contraste, el algoritmo propuesto aprovecha ambas direcciones, reparte mejor el tráfico y reduce la demora media en la mayoría de los escenarios evaluados, con mejoras muy marcadas en el Caso 1 y en el Caso 2 para cargas medias y bajas.

Además, la decisión basada en distancia y congestión evita bucles y mantiene el costo de decisión en tiempo constante, por lo que la estrategia escala mejor que el baseline cuando la red crece. Como limitación, ambos enfoques siguen siendo vulnerables a fallas de nodos críticos porque no incorporan mecanismos de reconvergencia ni descubrimiento dinámico.

References

- [1] A. Varga and R. Hornig, “OMNeT++,” in *Proc. Int. Conf. Simulation Tools and Techniques (Simutools)*, 2008.
- [2] Cátedra de Redes y Sistemas Distribuidos, “Laboratorio 4: kickstarter OMNeT++,” FaMAF/UNC, 2026.
- [3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2011.